Python Built-in Functions & Methods

Complete practical reference for List, Tuple, Dictionary, String, Set, and common built-in functions — with examples.

1. LIST

A list is ordered, mutable (changeable), and allows duplicate values.

```
numbers = [10, 20, 30, 20, 40]
```

Method	Use	Example
append()	Add one item at end	numbers.append(50)
extend()	Add multiple items	numbers.extend([60, 70])
insert()	Add at specific position	numbers.insert(1, 15)
remove()	Remove specific value	numbers.remove(20)
pop()	Remove and return item	numbers.pop()
clear()	Remove all items	numbers.clear()
index()	Find position	numbers.index(30)
count()	Count occurrences	numbers.count(20)
sort()	Sort list	numbers.sort()
reverse()	Reverse list	numbers.reverse()
copy()	Copy list	new = numbers.copy()

List examples

```
numbers = [10, 20, 30]
numbers.append(40)
print(numbers) # [10, 20, 30, 40]

numbers.extend([50, 60])
numbers.insert(1, 15)
numbers.remove(20)

x = numbers.pop()
print(x) # removed value
print(numbers) # remaining list

numbers.sort()
numbers.reverse()
```

Built-in functions used with Lists

```
numbers = [10, 20, 30, 40, 50]

print(len(numbers)) # 5
print(max(numbers)) # 50
print(min(numbers)) # 10
print(sum(numbers)) # 150
print(sorted(numbers)) # sorted copy

list("hello") # ['h', 'e', 'l', 'l', 'o']
list(range(5)) # [0, 1, 2, 3, 4]
any([False, False, True]) # True
all([True, True, True]) # True
```

2. TUPLE

A tuple is ordered and immutable (cannot be changed after creation).

```
numbers = (10, 20, 30, 20, 40)
```

Method	Use	Example
count()	Count occurrence	numbers.count(20)
index()	Find position	numbers.index(30)

```
numbers = (10, 20, 30, 20, 40)

print(numbers.count(20)) # 2
print(numbers.index(30)) # 2

print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
print(sorted(numbers))

list(numbers) # convert tuple to list
tuple([1, 2, 3]) # convert list to tuple
```

3. DICTIONARY

A dictionary stores data as key-value pairs.

```
student = {
"name": "Vishal",
"age": 23,
"course": "MCA"
}
```

Method	Use
keys()	Get all keys
values()	Get all values
items()	Get key-value pairs
get()	Get a value safely
update()	Add or update data
pop()	Remove a specific key
popitem()	Remove the last key-value pair
clear()	Remove everything
copy()	Copy dictionary
setdefault()	Get key or create it
fromkeys()	Create dictionary from keys

Dictionary examples

```
student = {"name": "Vishal", "age": 23}

print(student.keys())
print(student.values())
print(student.items())

print(student.get("name")) # Vishal
print(student.get("salary")) # None
print(student.get("salary", 0)) # 0

student.update({"age": 24})
student.update({"city": "Delhi"})

student.pop("age")
```

```
student.popitem()

student.setdefault("country", "India")

keys = ["name", "age", "city"]
student = dict.fromkeys(keys)
# {'name': None, 'age': None, 'city': None}

student = dict.fromkeys(keys, "Unknown")
```

Built-in functions with Dictionary

```
student = {"name": "Vishal", "age": 23, "marks": 85}

print(len(student)) # 3
print(max(student)) # marks (compares keys)
print(min(student)) # age
print(sorted(student)) # ['age', 'marks', 'name']
```

4. STRING

A string is a sequence of characters and is immutable.

```
text = "Hello Python"
```

Method	Purpose	Example
upper()	Convert to uppercase	"hello".upper()
lower()	Convert to lowercase	"HELLO".lower()
capitalize()	Capitalize first character	"hello".capitalize()
title()	Title case	"hello python".title()
swapcase()	Swap upper/lower case	"Hello".swapcase()
casefold()	Aggressive lowercase conversion	"HELLO".casefold()
find()	Find substring; returns -1 if absent	text.find("Python")
index()	Find substring; raises error if absent	text.index("Python")
count()	Count occurrences	text.count("hello")
startswith()	Check beginning	text.startswith("Hello")
endswith()	Check ending	text.endswith("Python")
replace()	Replace text	text.replace("Java","Python")
strip()	Remove outer whitespace	text.strip()
lstrip()	Remove left whitespace	text.lstrip()
rstrip()	Remove right whitespace	text.rstrip()
split()	Split into list	text.split()
join()	Join iterable into string	"-".join(items)
center()	Center text	text.center(10)
ljust()	Left justify	text.ljust(10)
rjust()	Right justify	text.rjust(10)
partition()	Split into 3 parts around separator	text.partition("=")
removeprefix()	Remove prefix if present	text.removeprefix("Hi")
removesuffix()	Remove suffix if present	text.removesuffix("!")
zfill()	Pad with zeros on left	"42".zfill(5)

String checking methods

Method	Checks
isalpha()	Only alphabetic characters
isdigit()	Only digits
isdecimal()	Decimal characters
isnumeric()	Numeric characters
isalnum()	Alphabets + numbers
isspace()	Only whitespace
islower()	All cased characters lowercase
isupper()	All cased characters uppercase
istitle()	Title case
isascii()	ASCII characters
isidentifier()	Valid Python identifier
isprintable()	Printable characters

```
text = "Python123"

print(text.isalpha()) # False
print(text.isdigit()) # False
print(text.isalnum()) # True

print("Hello".isalpha()) # True
print("123".isdigit()) # True
print("Hello123".isalnum())# True
print(" ".isspace()) # True
print("hello".islower()) # True
print("HELLO".isupper()) # True
```

String splitting and joining

```
text = "Python Java C++"
print(text.split())
# ['Python', 'Java', 'C++']

text = "apple,banana,mango"
print(text.split(","))
# ['apple', 'banana', 'mango']

items = ["Python", "Java", "C++"]
result = "-".join(items)
print(result)
# Python-Java-C++
```

5. SET

A set is unordered, mutable, and does not allow duplicate values.

```
numbers = {10, 20, 30, 20}
print(numbers)
# {10, 20, 30}
```

Method	Use
add()	Add one item
update()	Add multiple items
remove()	Remove item; error if absent
discard()	Remove item safely

Method	Use
pop()	Remove an arbitrary item
clear()	Remove all items
copy()	Copy set
union()	Combine sets
intersection()	Common elements
difference()	Elements in first set only
symmetric_difference()	Elements in either set but not both
issubset()	Check subset
issuperset()	Check superset
isdisjoint()	Check that there are no common elements

Set examples

```
numbers = {10, 20, 30}

numbers.add(40)
numbers.update([50, 60, 70])

numbers.remove(20) # error if 20 is absent
numbers.discard(20) # no error if 20 is absent

numbers.pop() # removes an arbitrary item
numbers.clear() # removes all items
```

Set operations

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

A.union(B) # {1, 2, 3, 4, 5, 6}
A | B

A.intersection(B) # {3, 4}
A & B

A.difference(B) # {1, 2}
A - B

A.symmetric_difference(B) # {1, 2, 5, 6}
A ^ B
```

Subset, superset, and disjoint

```
A = {1, 2}
B = {1, 2, 3, 4}

print(A.issubset(B)) # True
print(B.issuperset(A)) # True

A = {1, 2}
B = {3, 4}
print(A.isdisjoint(B)) # True
```

6. COMMON BUILT-IN FUNCTIONS

These built-in functions are useful with many Python collections and iterables.

Function	Purpose
len()	Number of elements
max()	Maximum value
min()	Minimum value
sum()	Total of numeric values
sorted()	Returns a sorted list
reversed()	Returns a reverse iterator
any()	True if at least one element is truthy
all()	True if every element is truthy
enumerate()	Provides index and value
zip()	Combines multiple iterables
type()	Checks data type
list()	Converts to list
tuple()	Converts to tuple
set()	Converts to set
dict()	Creates/converts dictionary

enumerate()

```
names = ["Amit", "Vishal", "Rahul"]

for index, name in enumerate(names):
print(index, name)

# 0 Amit
# 1 Vishal
# 2 Rahul
```

zip()

```
names = ["Amit", "Vishal", "Rahul"]
marks = [80, 90, 85]

for name, mark in zip(names, marks):
print(name, mark)

# Amit 80
# Vishal 90
# Rahul 85
```

7. QUICK REVISION CHART

Data Type	Important Methods
List	append, extend, insert, remove, pop, sort, reverse, count, index, clear, copy
Tuple	count, index
Dictionary	keys, values, items, get, update, pop, popitem, clear, copy, setdefault, fromkeys
String	upper, lower, split, join, replace, strip, find, index, count, startswith, endswith, isalpha, isdigit, isalnum
Set	add, update, remove, discard, pop, clear, copy, union, intersection, difference, symmetric_difference, issubset, issuperset, is

8. MOST IMPORTANT FOR INTERVIEWS & EXAMS

```
LIST
append, extend, insert, remove, pop, sort, reverse

TUPLE
count, index

DICTIONARY
keys, values, items, get, update, pop, setdefault

STRING
upper, lower, split, join, replace, strip
find, index, count
isalpha, isdigit, isalnum

SET
add, update, remove, discard
union, intersection, difference
symmetric_difference
issubset, issuperset
```

Tip: Learn the method purpose, syntax, return value, and one practical example for each method.